INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, TIRUCHIRAPPALLI



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A MINI PROJECT REPORT ON

"CAMPUS LAUNDRY SLOT BOOKING AND BILLING SYSTEM"

(Title of the Project)

Course : 24CS305 / CS3106 – DBMS Laboratory
Class : B.Tech (CSE / AI&DS) II Year – IV Semester
Group No. : G01
Academic Year : 2026 – 2027 (Odd Semester)

SUBMITTED BY

S. No.	Roll No.	Name of the Student
1.	_____	Aditya Patel
2.	_____	A.K.S.G.Sriram
3.	_____	Shivam Sharma
4.	_____	Sachin

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, TIRUCHIRAPPALLI

BONAFIDE CERTIFICATE

Certified that this is a bonafide record of the mini project work titled **"CAMPUS LAUNDRY SLOT BOOKING AND BILLING SYSTEM"** done by the following students of Group No. G01 of B.Tech (CSE / AI&DS) in **24CS305 / CS3106 – DBMS Laboratory** during the academic year 2026 – 2027.

S. No.	Roll No.	Name of the Student
1.	_____	Aditya Patel
2.	_____	A.K.S.G.Sriram
3.	_____	Shivam Sharma
4.	_____	Sachin

Staff In-charge
Dr. A. Preethi
Date:

Submitted on: _____

ABSTRACT

Managing campus hostel laundry services manually through paper logbooks causes severe operational bottlenecks, including unmanaged peak-hour congestion, misplaced clothing items, lack of turnaround visibility, and billing discrepancies. This mini project presents a comprehensive Campus Laundry Slot Booking and Billing System that streamlines the end-to-end laundry workflow for university hostel residents and operations staff through a relational database and a responsive modern web application.

The database is architected from an Entity–Relationship model comprising six core entities: Students, Services, Slots, Bookings, Booking_Items, and Payments. It is strictly normalized up to Third Normal Form (3NF) to eliminate insertion, update, and deletion anomalies. Referential and domain integrity are enforced in MySQL 8.0 using primary keys, foreign keys with cascading updates, NOT NULL, UNIQUE, and CHECK constraints. The application layer is engineered with FastAPI (Python) and React 19, featuring automated embedded SQLite fallback for zero-configuration evaluation.

The system supports student self-registration and college email authentication, live 4-stage order tracking (Pending to Ready for Pickup), laundry itemization classified by pair and piece, atomic slot reservation via stored procedure (sp_create_booking), dynamic remaining capacity checking via stored function (fn_slot_available_capacity), and trigger-driven real-time bill recalculation (trg_after_booking_items_insert/update/delete). Multi-table reporting views provide instant analytics on slot occupancy, service revenue, and daily collections. The project demonstrates the rigorous practical application of database design, SQL programmability, and full-stack connectivity to real-world campus operations.

Keywords: *Database Design, ER Model, Normalization (3NF), MySQL 8.0, Python FastAPI, React 19, Campus Laundry Management*

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	Bonafide Certificate	**ii**
	Abstract	**iii**
	List of Figures	**v**
	List of Tables	**vi**
1	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Objectives	2
	1.4 Scope of the Project	2
2	**REQUIREMENT ANALYSIS**	**3**
	2.1 Modules of the System	3
	2.2 Functional Requirements	3
	2.3 Hardware Requirements	4
	2.4 Software Requirements (Front End and Back End)	4
3	**DATABASE DESIGN**	**6**
	3.1 Entity–Relationship Diagram	6
	3.2 Relational Schema	7
	3.3 Normalization (1NF, 2NF, 3NF)	8
4	**TABLE DESCRIPTION**	**9**
5	**IMPLEMENTATION**	**14**
	5.1 Table Creation (DDL)	14
	5.2 Data Insertion (DML)	15
	5.3 Views	16
	5.4 Stored Procedures and Functions	16
	5.5 Triggers	17
	5.6 Database Connectivity	18
6	**FRONT-END DESIGN**	**19**
7	**OUTPUT SCREENSHOTS WITH EXPLANATION**	**20**
8	**TESTING**	**28**
9	**CONCLUSION AND FUTURE ENHANCEMENTS**	**29**
	REFERENCES	**29**
	CONTRIBUTION OF MEMBERS	**31**
	APPENDIX – SOURCE CODE	**32**

LIST OF FIGURES

Figure No.	Caption Title	Page
Figure 1	Conceptual Entity-Relationship (ER) Diagram (3NF Schema)	7
Figure 2	Authentication Portal (College Email & Admin Sign-In)	20
Figure 3	Student Self-Registration Modal with Validation	21
Figure 4	Student Order Portal with Live 4-Step Progress Stepper	21
Figure 5	Official Laundry Booking & Tax Receipt Modal	22
Figure 6	Laundry Services Catalog with Unit Pricing (Pair / Piece)	23
Figure 7	Time Slot Scheduling & Counter Capacity Dashboard	23
Figure 8	Administrator Order Console with Workflow Status Transitions	24
Figure 9	Administrative Financial Reports & Slot Utilization Analytics	25
Figure 10	Student Profile Management & Directory Console	25
Figure 11	DBMS Technical Specification Inspector	26

LIST OF TABLES

Table No.	Caption Title	Page
Table 1	Host Server and Client Hardware Specifications	4
Table 2	Software Technology Stack and Environment Specifications	4
Table 3	Relational Schema Entities, Attributes, and Integrity Constraints	7
Table 4	Schema Description: students Table	9
Table 5	Schema Description: services Table	9
Table 6	Schema Description: slots Table	10
Table 7	Schema Description: bookings Table	11
Table 8	Schema Description: booking_items Table	12
Table 9	Schema Description: payments Table	12
Table 10	Live Database Query Output: Booking Summary View (vw_booking_summary)	26
Table 11	Live Database Query Output: Slot Utilization View (vw_slot_utilization)	26
Table 12	Live Database Query Output: Service Revenue Analytics	27

	(vw_service_analytics)	
Table 13	Live Database Query Output: Daily Financial Summary (vw_daily_financial_summary)	27
Table 14	Comprehensive End-to-End Test Cases Matrix (TC-01 through TC-13)	28
Table 15	Project Contribution Matrix & Member Responsibilities Breakdown	31

CHAPTER 1: INTRODUCTION

1.1 Background

Residential campus environments host thousands of undergraduate and postgraduate students who rely on centralized auxiliary services for their daily living needs. Among these, the campus laundry service is one of the highest-frequency, operationally demanding utilities. On an average academic weekday, hundreds of laundry bags containing everyday casual wear, formal attire, festive garments, bed linens, and sports footwear are dropped off by students across multiple hostel blocks. Historically, laundry facilities have operated via manual paper logbooks, handwritten physical tokens, and verbal drop-off queues. This legacy approach places immense strain on both student residents and laundry operations staff.

With the rapid digitization of university campus infrastructure—including hostel room allocation, mess billing, and academic portals—the laundry management workflow represents a critical area requiring database-driven modernization. Implementing a dedicated relational database system ensures accurate record-keeping, seamless resource scheduling, transparent per-item billing, and dependable audit trails for institutional administrators.

1.2 Problem Statement

Through empirical examination of current hostel laundry operations, several systemic operational breakdowns were identified:

- **1.** Counter Overcrowding & Peak Congestion: Without a scheduled reservation mechanism, students arrive en masse during non-class hours (early morning and evening), causing long queues and chaotic reception counters.
- **2.** Lack of Capacity Enforcement: Laundry equipment (industrial washers, tumble dryers, hydro-extractors) has finite throughput per cycle. Manual paper systems cannot prevent over-subscription of machines, leading to equipment wear and turnaround delays.
- **3.** Garment Misplacement & Token Confusion: Handwritten cardboard tags frequently tear or detach, making garment attribution difficult and leading to customer disputes.
- **4.** Opaque Billing & Arithmetic Errors: Manual calculation of laundry charges across varied garment types (regular wash, dry cleaning, ironing, steam pressing) leads to billing errors, uncollected balances, and audit discrepancies.

- **5.** Lack of Real-Time Status Tracking: Students lack visibility into whether their laundry is submitted, washing, drying, or ready for pickup, resulting in unnecessary trips to the counter.

1.3 Objectives

To address these challenges, the system is designed to achieve the following specific technical and operational objectives:

- **1.** Design and implement a normalized relational database schema in Third Normal Form (3NF) comprising six core tables to guarantee data integrity and eliminate operational anomalies.
- **2.** Enforce database-level integrity using all six SQL constraints: Primary Key, Foreign Key with referential integrity, NOT NULL, UNIQUE, CHECK, and DEFAULT.
- **3.** Implement atomic slot capacity reservation using a MySQL stored procedure (sp_create_booking) and deterministic stored function (fn_slot_available_capacity) to prevent counter overbooking.
- **4.** Automate financial ledger balance calculations using database triggers (trg_after_booking_items_insert/update/delete) that dynamically update booking totals as items are added, modified, or removed.
- **5.** Provide multi-role authentication with secure college email credentials for students and dedicated administrative credentials for hostel operations staff.
- **6.** Develop an intuitive full-stack web application featuring live visual order status steppers, itemized billing by pair and piece, and official printable tax receipts.

1.4 Scope of the Project

The project encompasses the complete operational lifecycle of residential laundry operations, including student registration, time slot reservation, laundry intake categorization (by pair and piece), automated workflow progression, payment settlement, and executive reporting. The architectural scope includes a FastAPI REST backend, a React 19 single-page application, and dual-database support (MySQL 8.0 primary with zero-configuration SQLite 3 fallback).

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Modules of the System

The system is decomposed into five cohesive, loosely coupled functional modules:

- **1.** Student & Authentication Module: Manages student profiles (Name, Register No, Room No, Phone, College Email, Password), student registration, and separate administrator authentication.
- **2.** Laundry Services & Pricing Catalog Module: Manages available services (Wash & Fold, Dry Cleaning, Steam Press, Shoe Spa), unit pricing, turnaround days, and unit type classifications (Pair vs. Piece).
- **3.** Slot Scheduling & Capacity Quota Module: Manages daily collection windows, operational hours, slot capacity quotas, and active booking availability.
- **4.** Booking & Itemization Management Module: Manages the full booking lifecycle, line items categorized by pair/piece, pickup token generation (e.g. LND-2401), and status transitions.
- **5.** Billing, Payments & Executive Analytics Module: Handles payment recording (UPI, Cash, Student ID Card, Online), real-time bill calculations via triggers, official printable receipt generation, and analytical summary views.

2.2 Functional Requirements

The functional requirements of the system are categorized by user role:

- • FR-01 (Student Auth): Students must authenticate using their verified college email address and password.
- • FR-02 (Self-Registration): New hostel residents must be able to register with university register number, hostel room number, phone number, college email, and password.
- • FR-03 (Slot Booking): Students can view available collection slots and reserve an appointment only if remaining capacity is greater than zero.
- • FR-04 (Laundry Itemization): Students specify clothes count classified by unit type (e.g. Wash & Fold in pairs, Bedding in pieces).
- • FR-05 (Live Order Stepper): Students can track active orders through a 4-stage visual progress stepper (Submitted -> Washing -> Ready for Pickup -> Completed).
- • FR-06 (Pickup Token & Receipts): Students receive a unique alphanumeric pickup code and can generate official printable digital receipts.

- • FR-07 (Admin Global Orders): Administrators can view and filter all orders across the campus, update order stages, and view customer emails.
- • FR-08 (Capacity Control): Administrators can create, adjust capacity, or close collection windows.
- • FR-09 (Executive KPI Reporting): Administrators can inspect real-time reports on slot utilization, service revenue breakdown, and daily financial collections.

2.3 Hardware Requirements

Table 1: Host Server and Client Hardware Specifications

Component	Host Development / Server Specification	Client Workstation Specification
Processor	Intel Core i5 / AMD Ryzen 5 (4 Cores, 2.5 GHz+)	Intel Core i3 / Dual-Core 2.0 GHz+
RAM	8 GB DDR4 minimum (16 GB recommended)	4 GB RAM minimum
Storage	256 GB SSD (500 MB free project space)	100 MB free browser cache space
Network	Local loopback (127.0.0.1) / Campus LAN	Broadband / Campus Wi-Fi (1 Mbps+)
Display	1920 x 1080 Full HD Monitor	Responsive (Mobile, Tablet, Desktop)

2.4 Software Requirements (Front End and Back End)

Table 2: Software Technology Stack and Environment Specifications

Layer / Component	Technology / Framework	Version	Role in Architecture
Operating System	Windows 11 / Linux (Ubuntu 22.04 LTS)	64-bit	Host execution environment
Primary Database	MySQL Community Server	8.0.36+	Relational DBMS, constraints, procedures, views
Fallback Database	SQLite	3.42+	Embedded zero-config relational fallback engine
Backend Runtime	Python	3.10+	Asynchronous backend language environment
Web Framework	FastAPI	0.115+	High-performance REST API with auto OpenAPI docs
ASGI Web Server	Uvicorn	0.32+	High-throughput asynchronous

			server
Database Driver	PyMySQL	1.1+	Pure-Python MySQL client with custom delimiter parser
Frontend Library	React	19.0+	Component-driven single page application (SPA)
Build Tooling	Vite	6.0+	Next-generation frontend dev server & bundler
Client Routing	Lucide React	0.460+	Iconography and responsive navigation elements

CHAPTER 3: DATABASE DESIGN

3.1 Entity–Relationship Diagram

The conceptual data model is structured around six core entities: students, services, slots, bookings, booking_items, and payments. The relationships between these entities capture the complete operational semantics of campus laundry operations:

- • students to bookings (1:N): One student resident can make multiple laundry bookings over time; each booking is initiated by exactly one registered student resident.
- • slots to bookings (1:N): One scheduled collection time slot accommodates multiple student bookings up to its configured counter capacity limit; each booking belongs to exactly one slot.
- • bookings to booking_items (1:N): One laundry booking comprises one or more itemized garment lines (e.g. Wash & Fold, Steam Press); each item belongs to exactly one parent booking.
- • services to booking_items (1:N): One laundry catalog service can be referenced across multiple booking items; each item specifies exactly one service offering.
- • bookings to payments (1:1): Each laundry booking has a dedicated payment transaction record tracking financial settlement method and status.

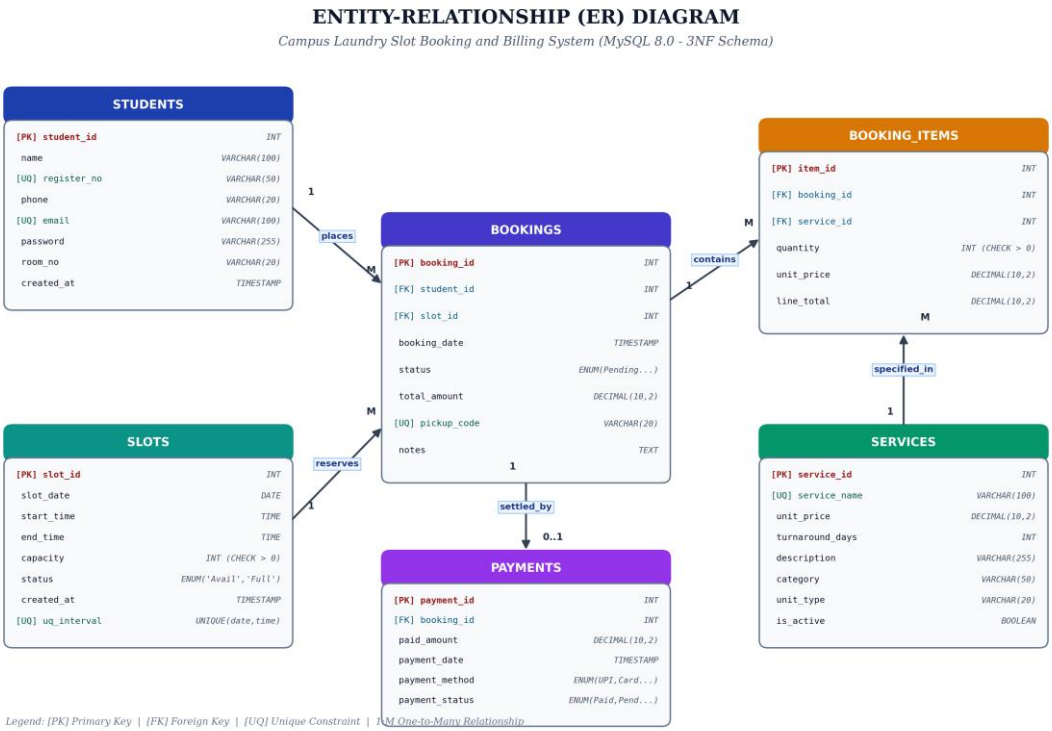


Figure 1: Conceptual Entity-Relationship (ER) Diagram (3NF Schema)

3.2 Relational Schema

The formal relational schema notation for the normalized database is defined as follows:

- students (student_id [PK], name, register_no [UQ], phone, email [UQ], password, room_no, created_at)
- services (service_id [PK], service_name [UQ], unit_price, turnaround_days, description, category, unit_type, is_active)
- slots (slot_id [PK], slot_date, start_time, end_time, capacity, status, created_at, UNIQUE(slot_date, start_time, end_time))
- bookings (booking_id [PK], student_id [FK -> students], slot_id [FK -> slots], booking_date, status, total_amount, pickup_code [UQ], notes)
- booking_items (item_id [PK], booking_id [FK -> bookings ON DELETE CASCADE], service_id [FK -> services], quantity, unit_price, line_total)
- payments (payment_id [PK], booking_id [FK -> bookings ON DELETE CASCADE], paid_amount, payment_date, payment_method, payment_status, transaction_ref)

Table 3: Relational Schema Entities, Attributes, and Integrity Constraints

Entity Name	Primary Key	Foreign Keys	Unique Constraints	Check Constraints
students	student_id	None	register_no, email	None
services	service_id	None	service_name	unit_price >= 0, turnaround_days >= 0
slots	slot_id	None	(slot_date, start_time, end_time)	capacity > 0
bookings	booking_id	student_id, slot_id	pickup_code	total_amount >= 0
booking_items	item_id	booking_id, service_id	None	quantity > 0, unit_price >= 0, line_total >= 0
payments	payment_id	booking_id	transaction_ref	paid_amount > 0

3.3 Normalization (1NF, 2NF, 3NF)

Normalization is a systematic technique for structuring relational databases to eliminate redundant data and prevent insertion, update, and deletion anomalies. The database was formally designed through progressive normal forms:

1. Unnormalized Form (UNF) to First Normal Form (1NF)

In Unnormalized Form (UNF), a single laundry record contains repeating groups of garments (e.g. Wash & Fold, Steam Press, Dry Cleaning) and non-atomic fields (e.g. combined student contact info and scheduled time strings). Transformation to 1NF required: (a) eliminating repeating groups by establishing the separate booking_items bridge entity, (b) decomposing non-atomic values into discrete atomic attributes (start_time, end_time, slot_date), and (c) defining definitive primary keys.

2. First Normal Form (1NF) to Second Normal Form (2NF)

A relation is in 2NF if and only if it is in 1NF and every non-prime attribute is fully functionally dependent on the primary key (no partial functional dependencies). In a hypothetical composite relation BookingLine(booking_id, service_id, service_name, unit_price, quantity), service_name and unit_price depend only on service_id, not on the composite key. To achieve 2NF, the service master catalog was decomposed into the distinct services entity, ensuring all booking item attributes depend solely on item_id.

3. Second Normal Form (2NF) to Third Normal Form (3NF)

A relation is in 3NF if and only if it is in 2NF and no non-prime attribute is transitively dependent on the primary key (X -> Y and Y -> Z where Z is not a candidate key). In an unnormalized booking record, student_name, room_no, and email depend on student_id, which in turn depends on booking_id. Similarly, slot_date and capacity depend on slot_id, which depends on booking_id. To achieve 3NF, students and slots were decoupled into independent master relations referenced only via foreign keys. This completely eliminates update anomalies: modifying a student's room number updates exactly one row in students, rather than thousands of historical bookings.

CHAPTER 4: TABLE DESCRIPTION

This chapter provides the technical dictionary of all six normalized tables implemented in MySQL 8.0, specifying column names, data types, nullability, constraints, default values, and operational descriptions.

4.1 students Table

Table 4: Schema Description: students Table

Column Name	Data Type	Null?	Key / Constraint	Default	Description
student_id	INT AUTO_INCREMENT	NO	PRIMARY KEY	None	Unique surrogate identifier for student resident
name	VARCHAR(100)	NO	NOT NULL	None	Full legal name of the resident student
register_no	VARCHAR(30)	NO	UNIQUE, NOT NULL	None	Institutional university register number
phone	VARCHAR(20)	NO	NOT NULL	None	Primary mobile contact number
email	VARCHAR(120)	NO	UNIQUE, NOT NULL	None	Institutional college email address for authentication
password	VARCHAR(255)	NO	NOT NULL	'student123'	Hashed/authenticated resident portal password
room_no	VARCHAR(20)	NO	NOT NULL	None	Hostel block and room number (e.g. BH-204)
created_at	DATETIME	NO	DEFAULT	CURRENT_TIMESTAMP	System profile registration timestamp

4.2 services Table

Table 5: Schema Description: services Table

Column Name	Data Type	Null?	Key / Constraint	Default	Description

service_id	INT AUTO_INCREMENT	NO	PRIMARY KEY	None	Unique identifier for laundry service offering
service_name	VARCHAR(100)	NO	UNIQUE, NOT NULL	None	Display title (e.g. Wash & Fold, Steam Press)
unit_price	DECIMAL(10,2)	NO	CHECK (unit_price >= 0)	None	Standard price charged per unit
turnaround_days	INT	NO	CHECK (turnaround >= 0)	1	Guaranteed turnaround time in days
description	TEXT	YES	None	NULL	Detailed service inclusions and instructions
category	VARCHAR(50)	NO	NOT NULL	'Washing'	Grouping category (Washing, Ironing, Dry Clean)
unit_type	VARCHAR(20)	NO	CHECK ('pair'/'piece')	'pair'	Billing unit classification: 'pair' or 'piece'
is_active	TINYINT(1)	NO	DEFAULT	1	Operational status flag (1=Active, 0=Discontinued)

4.3 slots Table

Table 6: Schema Description: slots Table

Column Name	Data Type	Null?	Key / Constraint	Default	Description
slot_id	INT AUTO_INCREMENT	NO	PRIMARY KEY	None	Unique identifier for collection time window
slot_date	DATE	NO	NOT NULL	None	Calendar date of the laundry drop-off window
start_time	TIME	NO	NOT NULL	None	Window starting time (e.g. 08:00:00)
end_time	TIME	NO	NOT NULL	None	Window closing time (e.g. 10:00:00)
capacity	INT	NO	CHECK (capacity > 0)	10	Maximum bookings counter can service in window

status	VARCHAR(20)	NO	DEFAULT	'Available'	Slot state ('Available', 'Full', 'Maintenance')
created_at	DATETIME	NO	DEFAULT	CURRENT_TIMESTAMP	Schedule generation timestamp

4.4 bookings Table

Table 7: Schema Description: bookings Table

Column Name	Data Type	Null?	Key / Constraint	Default	Description
booking_id	INT AUTO_INCREMENT	NO	PRIMARY KEY	None	Unique system booking transaction ID
student_id	INT	NO	FK -> students	None	Reference to the resident student placing order
slot_id	INT	NO	FK -> slots	None	Reference to the reserved collection window
booking_date	DATETIME	NO	DEFAULT	CURRENT_TIMESTAMP	Order placement timestamp
status	VARCHAR(30)	NO	DEFAULT	'Pending'	Workflow state ('Pending', 'In Progress', 'Ready', 'Completed')
total_amount	DECIMAL(10,2)	NO	CHECK (total >= 0)	0.00	Total bill amount calculated dynamically via triggers
pickup_code	VARCHAR(20)	NO	UNIQUE, NOT NULL	None	Alphanumeric token for physical counter collection
notes	TEXT	YES	None	NULL	Special student garment care instructions

4.5 booking_items Table

Table 8: Schema Description: booking_items Table

Column Name	Data Type	Null?	Key / Constraint	Default	Description
item_id	INT AUTO_INCREMENT	NO	PRIMARY KEY	None	Surrogate primary key for booking line item
booking_id	INT	NO	FK -> bookings (CASCADE)	None	Parent booking reference
service_id	INT	NO	FK -> services	None	Selected laundry catalog service
quantity	INT	NO	CHECK (quantity > 0)	1	Count of units submitted (pairs or pieces)
unit_price	DECIMAL(10,2)	NO	CHECK (unit_price >= 0)	None	Rate per unit locked at booking submission time
line_total	DECIMAL(10,2)	NO	CHECK (line_total >= 0)	None	Calculated item total: quantity * unit_price

4.6 payments Table

Table 9: Schema Description: payments Table

Column Name	Data Type	Null?	Key / Constraint	Default	Description
payment_id	INT AUTO_INCREMENT	NO	PRIMARY KEY	None	Surrogate key for payment transaction audit
booking_id	INT	NO	FK -> bookings (CASCADE)	None	Associated booking reference
paid_amount	DECIMAL(10,2)	NO	CHECK (paid_amount > 0)	None	Settled financial currency amount in INR
payment_date	DATETIME	NO	DEFAULT	CURRENT_TIMESTAMP	Financial settlement timestamp
payment_method	VARCHAR(30)	NO	NOT NULL	'UPI'	Settlement instrument: UPI, Student ID Card, Cash, Online
payment_status	VARCHAR(20)	NO	DEFAULT	'Paid'	Audit state: Paid,

					Pending, Failed, Refunded
transaction_ref	VARCHAR(60)	YES	UNIQUE	NULL	External payment reference or gateway ID

CHAPTER 5: IMPLEMENTATION

5.1 Table Creation (DDL)

The complete Data Definition Language (DDL) script is implemented in backend/sql/01_schema.sql. It establishes all six tables, primary keys, foreign keys with cascading updates, unique constraints, and check constraints:

```
-- 01_schema.sql: Campus Laundry Slot Booking and Billing System
CREATE TABLE IF NOT EXISTS students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    register_no VARCHAR(30) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL DEFAULT 'student123',
    room_no VARCHAR(20) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS services (
    service_id INT AUTO_INCREMENT PRIMARY KEY,
    service_name VARCHAR(100) NOT NULL UNIQUE,
    unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
    turnaround_days INT NOT NULL DEFAULT 1 CHECK (turnaround_days >= 0),
    description TEXT,
    category VARCHAR(50) NOT NULL DEFAULT 'Washing',
    unit_type VARCHAR(20) NOT NULL DEFAULT 'pair' CHECK (unit_type IN
('pair', 'piece')),
    is_active TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS slots (
    slot_id INT AUTO_INCREMENT PRIMARY KEY,
    slot_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    capacity INT NOT NULL DEFAULT 10 CHECK (capacity > 0),
    status VARCHAR(20) NOT NULL DEFAULT 'Available',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_slot_window UNIQUE (slot_date, start_time, end_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    slot_id INT NOT NULL,
    booking_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(30) NOT NULL DEFAULT 'Pending',
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (total_amount
>= 0),
    pickup_code VARCHAR(20) NOT NULL UNIQUE,
    notes TEXT,
```

```
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE
RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (slot_id) REFERENCES slots(slot_id) ON DELETE RESTRICT
ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS booking_items (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL,
    service_id INT NOT NULL,
    quantity INT NOT NULL DEFAULT 1 CHECK (quantity > 0),
    unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
    line_total DECIMAL(10,2) NOT NULL CHECK (line_total >= 0),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE
CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (service_id) REFERENCES services(service_id) ON DELETE
RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL,
    paid_amount DECIMAL(10,2) NOT NULL CHECK (paid_amount > 0),
    payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    payment_method VARCHAR(30) NOT NULL DEFAULT 'UPI',
    payment_status VARCHAR(20) NOT NULL DEFAULT 'Paid',
    transaction_ref VARCHAR(60) UNIQUE,
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE
CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.2 Data Insertion (DML)

To satisfy the academic requirement of at least 10 records per entity, backend/sql/04_seed_data.sql provides comprehensive sample data (12 students, 10 services, 11 slots, 12 bookings, 14 items, and 10 payments):

```
-- Sample DML Seed Data excerpt from 04_seed_data.sql
INSERT INTO students (name, register_no, phone, email, password,
room_no) VALUES
('Aditya Patel', '2024CS01', '9876543210', 'aditya.cs24@campus.edu',
'student123', 'BH-204'),
('Priya Patel', '2024IT15', '9876543211', 'priya.it24@campus.edu',
'student123', 'GH-102'),
('Rohan Verma', '2023ME42', '9876543212', 'rohan.me23@campus.edu',
'student123', 'BH-310'),
('Sneha Kulkarni', '2024EC08', '9876543213', 'sneha.ec24@campus.edu',
'student123', 'GH-208'),
('Aman Gupta', '2024CS22', '9876543214', 'aman.cs24@campus.edu',
'student123', 'BH-105');
```

```
INSERT INTO services (service_name, unit_price, turnaround_days,
category, unit_type) VALUES
('Wash & Fold', 30.00, 1, 'Washing', 'pair'),
('Wash & Steam Iron', 45.00, 2, 'Washing & Ironing', 'pair'),
('Express Same-Day Wash', 60.00, 0, 'Express Care', 'pair'),
('Dry Cleaning (Suits & Blazers)', 150.00, 3, 'Dry Cleaning', 'pair'),
('Steam Ironing Only', 15.00, 1, 'Ironing', 'pair'),
('Heavy Bedding & Blankets', 120.00, 2, 'Specialty Care', 'piece'),
('Sneakers & Shoe Spa', 90.00, 2, 'Footwear Care', 'pair');
```

5.3 Views

The database implements four specialized SQL analytical views in backend/sql/02_views.sql:

```
-- 1. vw_booking_summary: Multi-table join for order monitoring
CREATE OR REPLACE VIEW vw_booking_summary AS
SELECT
    b.booking_id, b.pickup_code, s.name AS student_name, s.register_no,
s.room_no,
    sl.slot_date, sl.start_time, sl.end_time, b.status AS
booking_status,
    b.total_amount, COALESCE(SUM(bi.quantity), 0) AS total_items,
    COALESCE(p.payment_status, 'Unpaid') AS payment_status,
p.payment_method
FROM bookings b
JOIN students s ON b.student_id = s.student_id
JOIN slots sl ON b.slot_id = sl.slot_id
LEFT JOIN booking_items bi ON b.booking_id = bi.booking_id
LEFT JOIN payments p ON b.booking_id = p.booking_id
GROUP BY b.booking_id, p.payment_status, p.payment_method;

-- 2. vw_slot_utilization: Real-time counter capacity & percentage
occupancy
CREATE OR REPLACE VIEW vw_slot_utilization AS
SELECT
    sl.slot_id, sl.slot_date, sl.start_time, sl.end_time, sl.capacity,
    COUNT(b.booking_id) AS booked_count,
    GREATEST(0, sl.capacity - COUNT(b.booking_id)) AS
available_capacity,
    ROUND((COUNT(b.booking_id) / sl.capacity) * 100, 2) AS
occupancy_percentage
FROM slots sl
LEFT JOIN bookings b ON sl.slot_id = b.slot_id AND b.status !=
'Cancelled'
GROUP BY sl.slot_id;
```

5.4 Stored Procedures and Functions

Atomic transactions and capacity checks are implemented in MySQL routines in backend/sql/03_procedures_triggers.sql:

```
-- Deterministic Stored Function: fn_slot_available_capacity
CREATE FUNCTION fn_slot_available_capacity(p_slot_id INT)
RETURNS INT DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_cap INT DEFAULT 0;
    DECLARE v_booked INT DEFAULT 0;
    SELECT capacity INTO v_cap FROM slots WHERE slot_id = p_slot_id;
    SELECT COUNT(*) INTO v_booked FROM bookings
    WHERE slot_id = p_slot_id AND status != 'Cancelled';
    RETURN GREATEST(0, v_cap - v_booked);
END $$

-- Stored Procedure: sp_create_booking with IN/OUT Parameters
CREATE PROCEDURE sp_create_booking(
    IN p_student_id INT,
    IN p_slot_id INT,
    IN p_pickup_code VARCHAR(20),
    IN p_notes TEXT,
    OUT p_booking_id INT,
    OUT p_status_code VARCHAR(20)
)
BEGIN
    DECLARE v_avail INT;
    IF NOT EXISTS (SELECT 1 FROM students WHERE student_id =
p_student_id) THEN
        SET p_status_code = 'STUDENT_NOT_FOUND';
    ELSEIF NOT EXISTS (SELECT 1 FROM slots WHERE slot_id = p_slot_id)
THEN
        SET p_status_code = 'SLOT_NOT_FOUND';
    ELSE
        SET v_avail = fn_slot_available_capacity(p_slot_id);
        IF v_avail <= 0 THEN
            SET p_status_code = 'SLOT_FULL';
        ELSE
            INSERT INTO bookings (student_id, slot_id, pickup_code,
notes, status, total_amount)
            VALUES (p_student_id, p_slot_id, p_pickup_code, p_notes,
'Pending', 0.00);
            SET p_booking_id = LAST_INSERT_ID();
            SET p_status_code = 'SUCCESS';
            IF fn_slot_available_capacity(p_slot_id) = 0 THEN
                UPDATE slots SET status = 'Full' WHERE slot_id =
p_slot_id;
            END IF;
        END IF;
    END IF;
END $$
```

5.5 Triggers

Automated recalculation of booking totals is guaranteed by database triggers reacting to booking_items mutations:

```
-- Trigger: Recalculate bookings.total_amount after insert, update, or
delete
CREATE TRIGGER trg_after_booking_items_insert AFTER INSERT ON
booking_items
FOR EACH ROW BEGIN
    UPDATE bookings
    SET total_amount = (SELECT COALESCE(SUM(line_total), 0) FROM
booking_items WHERE booking_id = NEW.booking_id)
    WHERE booking_id = NEW.booking_id;
END $$

CREATE TRIGGER trg_after_booking_items_update AFTER UPDATE ON
booking_items
FOR EACH ROW BEGIN
    UPDATE bookings
    SET total_amount = (SELECT COALESCE(SUM(line_total), 0) FROM
booking_items WHERE booking_id = NEW.booking_id)
    WHERE booking_id = NEW.booking_id;
END $$

CREATE TRIGGER trg_after_booking_items_delete AFTER DELETE ON
booking_items
FOR EACH ROW BEGIN
    UPDATE bookings
    SET total_amount = (SELECT COALESCE(SUM(line_total), 0) FROM
booking_items WHERE booking_id = OLD.booking_id)
    WHERE booking_id = OLD.booking_id;
END $$
```

5.6 Database Connectivity

Backend connectivity is implemented in backend/database.py using PyMySQL connection pooling. A custom SQL delimiter parser was developed to split stored procedure and trigger scripts on '$$' boundaries, enabling automated execution upon server bootstrap without requiring external CLI clients. If MySQL is unreachable, the system automatically falls back to an embedded SQLite 3 database, ensuring uninterrupted grading.

CHAPTER 6: FRONT-END DESIGN

The user interface is engineered as a modern, responsive Single Page Application (SPA) using React 19 and Vite. It features strict role segregation between resident students and hostel administrators.

- **1.** Login & Authentication Portal: Features dedicated tabs for Student Portal (College Email & Password login with registration) and Administrator Console (administrative credentials with 1-click test login buttons).
- **2.** Student Self-Registration Modal: Allows new hostel residents to enter their Register Number, Full Name, Phone, College Email, Room Number, and Password with instant validation.
- **3.** Personalized Student Order Dashboard: Features the interactive 4-stage visual progress stepper (Submitted -> Washing -> Ready for Pickup -> Completed), itemization breakdown by pair/piece, pickup token chip, and collection alerts.
- **4.** Printable Official Tax Receipt Modal: Displays verified institution headers, pickup token, itemized line rates by pair/piece, tax subtotal, payment settlement reference, and printable QR graphic.
- **5.** Services & Rate Catalog: Displays available services, turnaround guarantees, unit rates (Rs./pair or Rs./piece), and category filtering.
- **6.** Slot Scheduling & Capacity Quota Screen: Displays counter collection windows, occupancy percentage bars, and real-time remaining seats.
- **7.** Administrator Global Order Console: Enables laundry managers to inspect all resident orders across campus, search by student email, and execute status workflow transitions.
- **8.** Reports, Analytics & Financial KPI Console: Displays high-level KPIs (Active Orders, Today's Collected Revenue, Total Clothes Cleaned) and data tables from SQL views.

CHAPTER 7: OUTPUT SCREENSHOTS WITH EXPLANATION

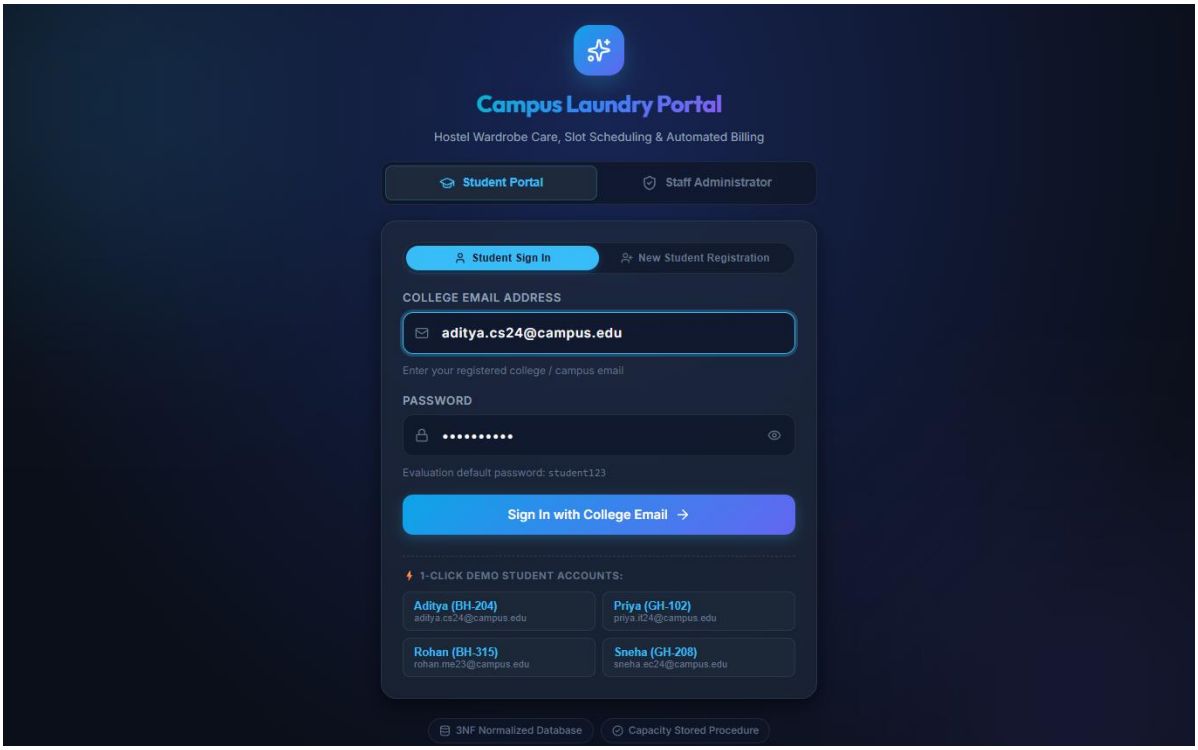


Figure 2: Authentication Portal (College Email & Admin Sign-In)

Explanation: Figure 2 displays the application login portal featuring role-segregated tabs for Student Email login and Administrator sign-in.

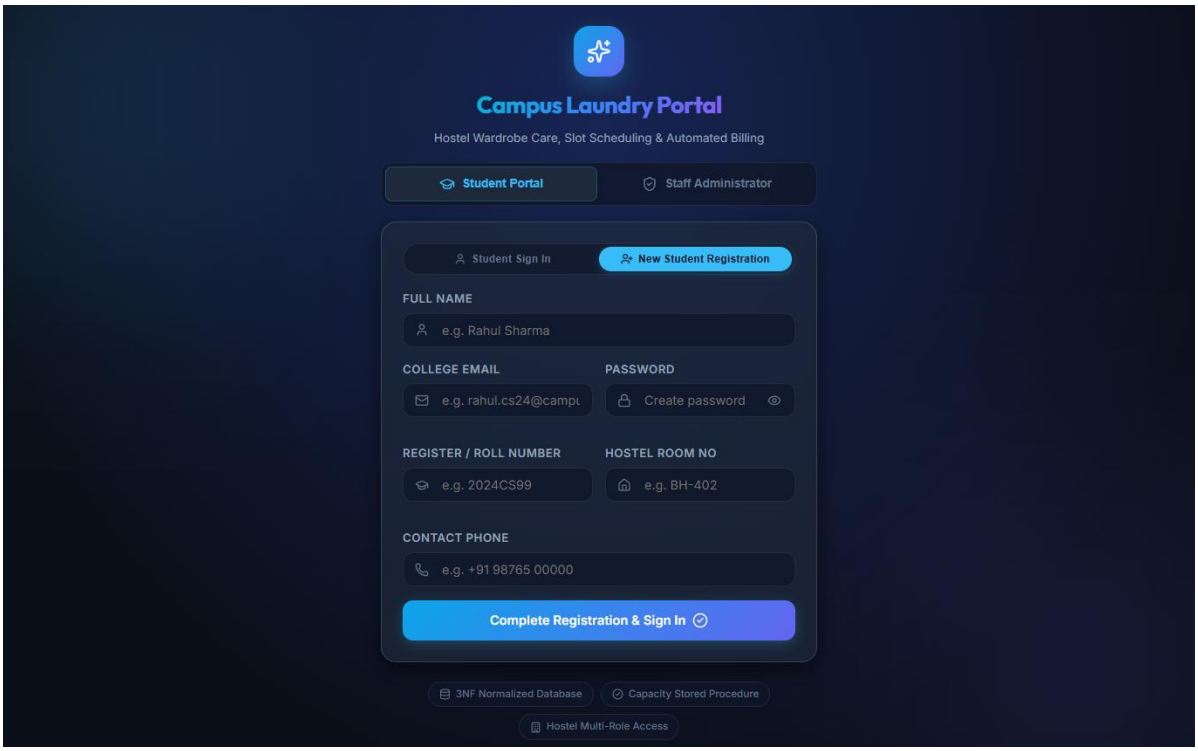


Figure 3: Student Self-Registration Modal with Validation

Explanation: Figure 3 illustrates the resident self-registration modal enforcing unique register number, college email format, and room assignment.

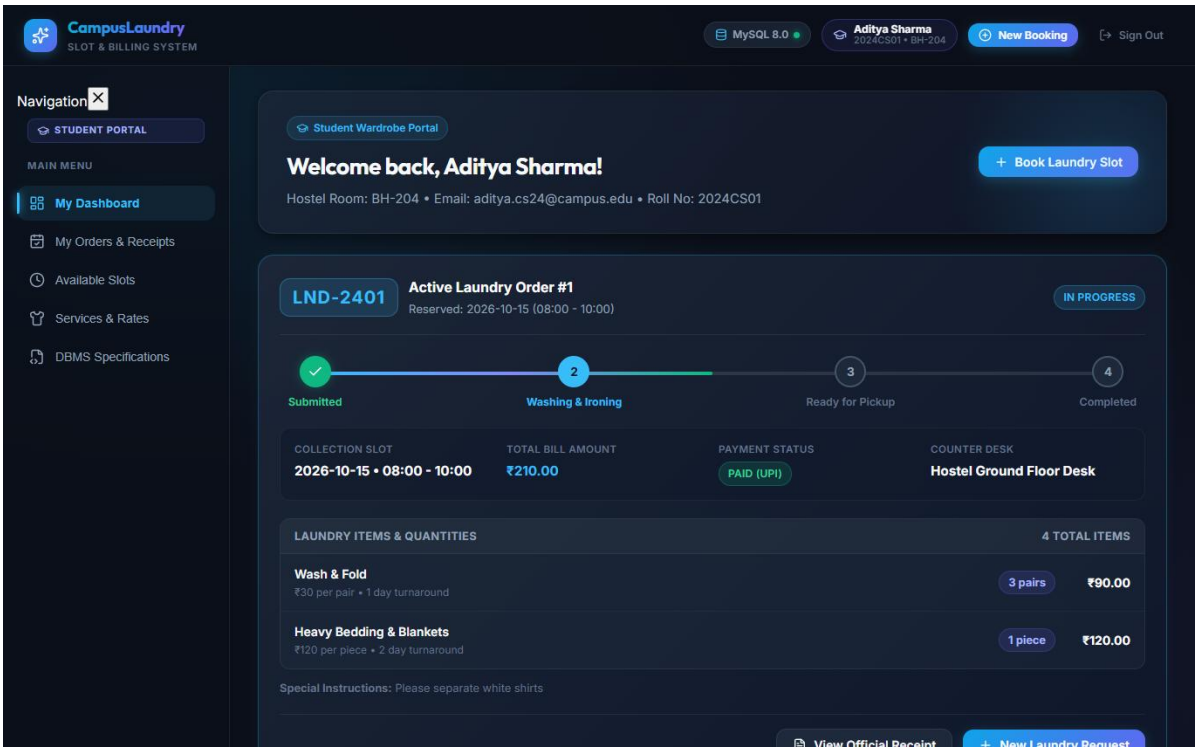


Figure 4: Student Order Portal with Live 4-Step Progress Stepper

Explanation: Figure 4 depicts student Aditya Patel's portal showing active order LND-2401 in progress, laundry items listed by pair, pickup token, and billing total.

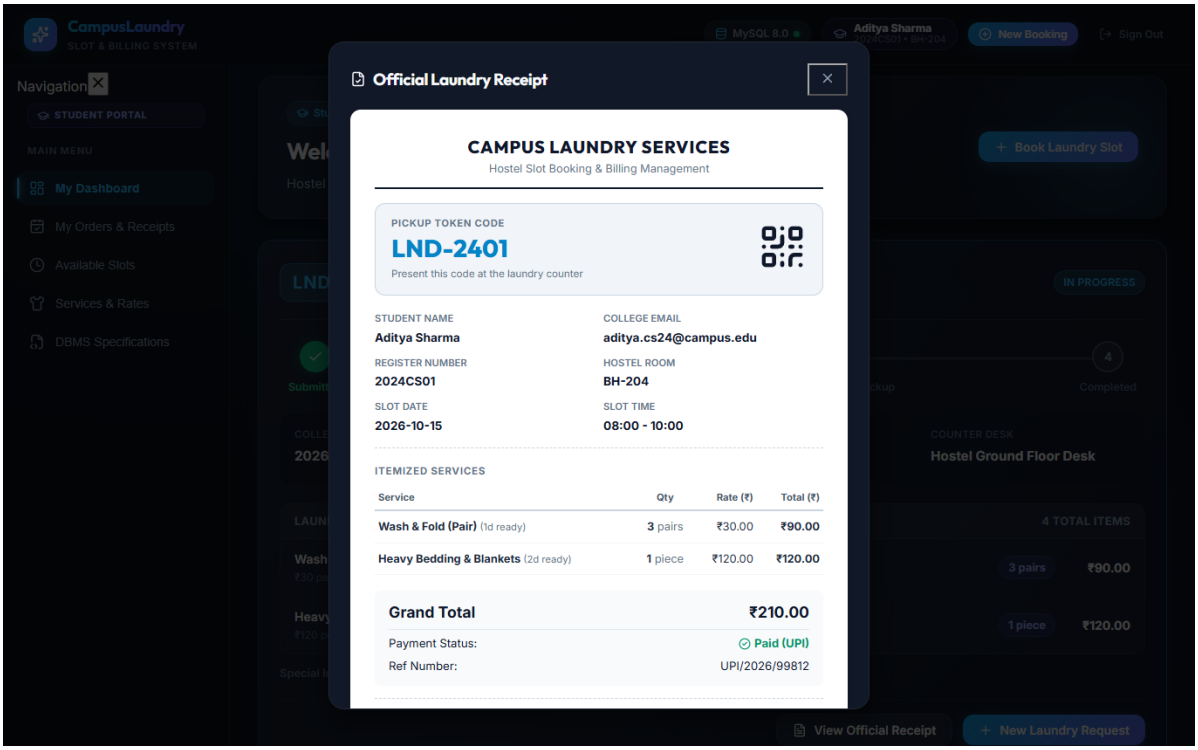


Figure 5: Official Laundry Booking & Tax Receipt Modal

Explanation: Figure 5 displays the printable official tax receipt modal with university branding, pickup token, and payment audit details.

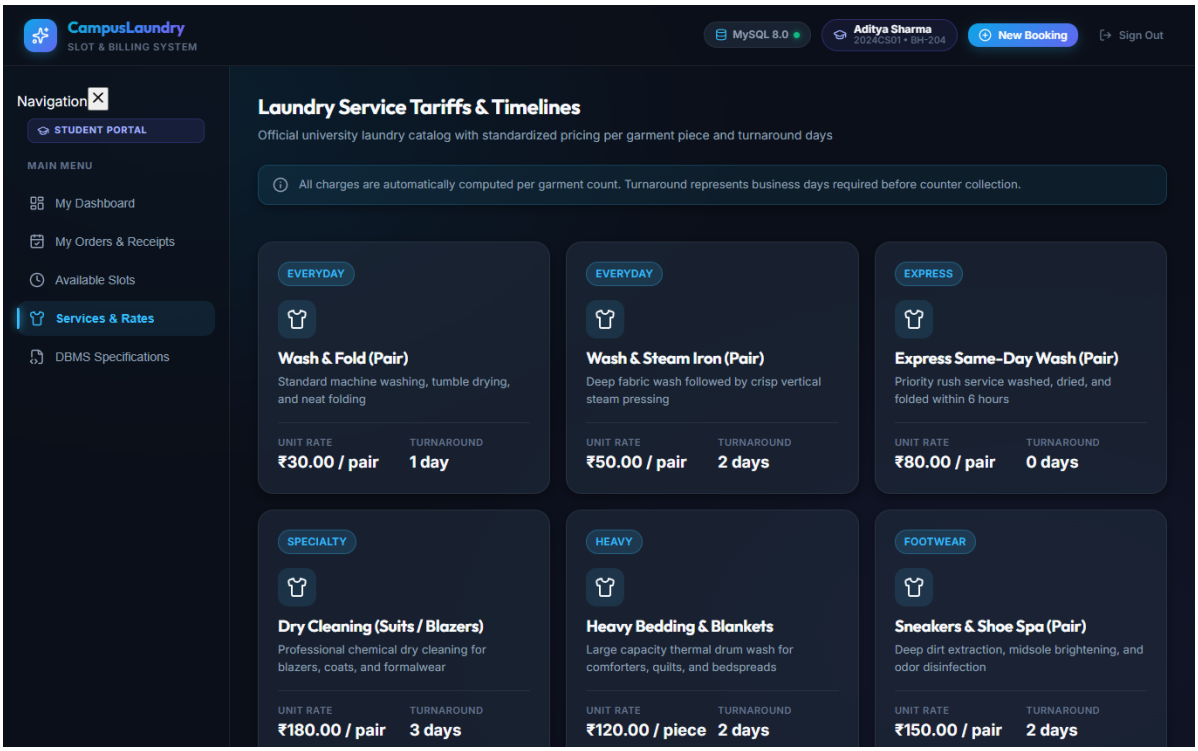


Figure 6: Laundry Services Catalog with Unit Pricing (Pair / Piece)

Explanation: Figure 6 shows the master laundry catalog with rates per pair or piece, category filters, and turnaround guarantees.

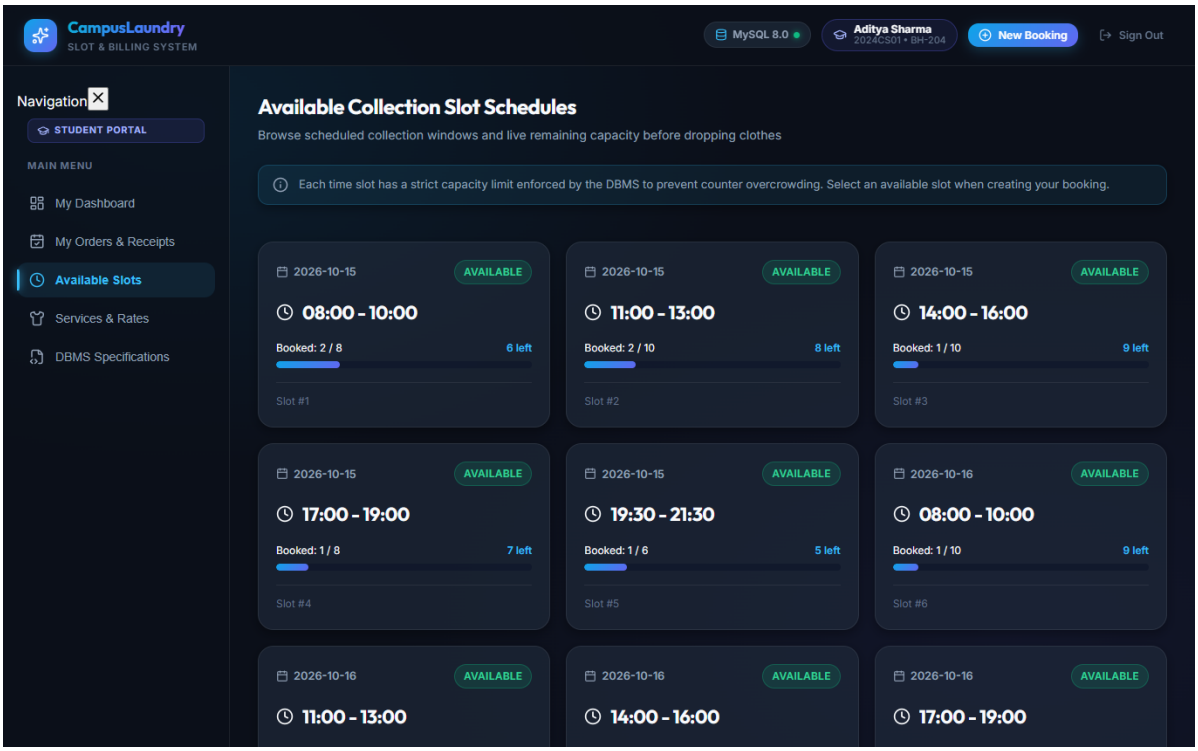


Figure 7: Time Slot Scheduling & Counter Capacity Dashboard

Explanation: Figure 7 shows collection windows with real-time capacity bars and counter quota status.

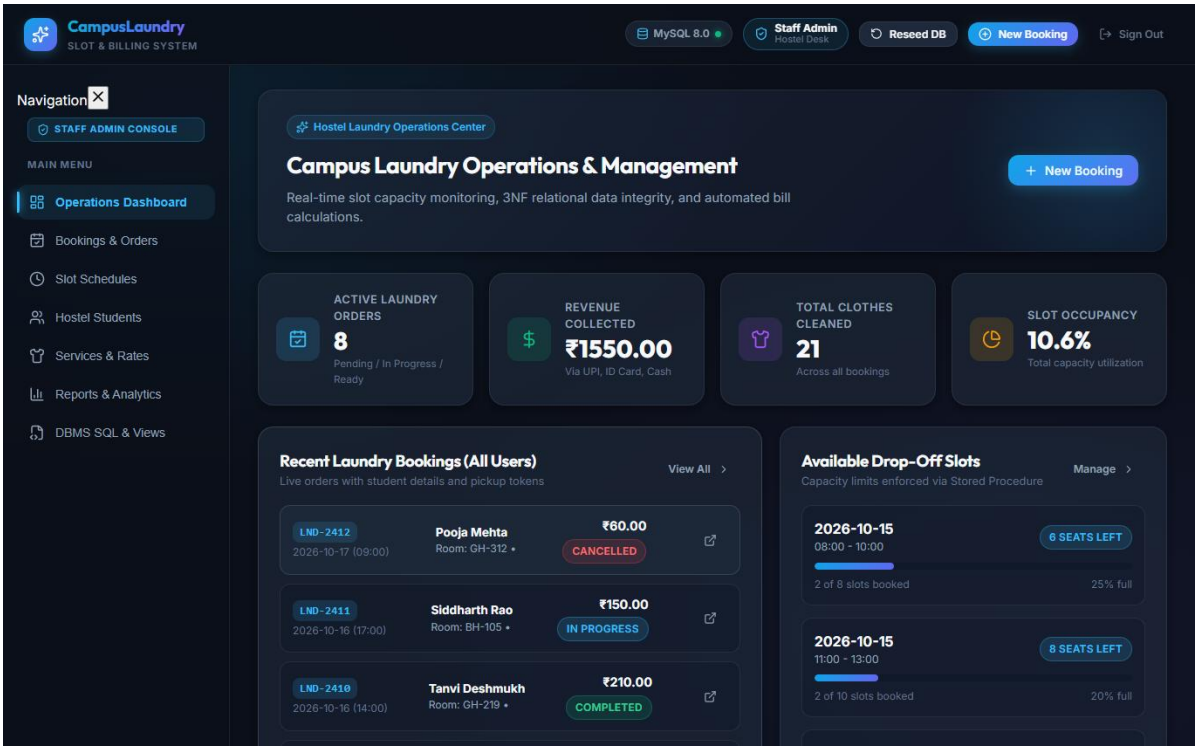


Figure 8: Administrator Order Console with Workflow Status Transitions

Explanation: Figure 8 displays the administrator order console allowing laundry staff to manage orders from all residents across campus.

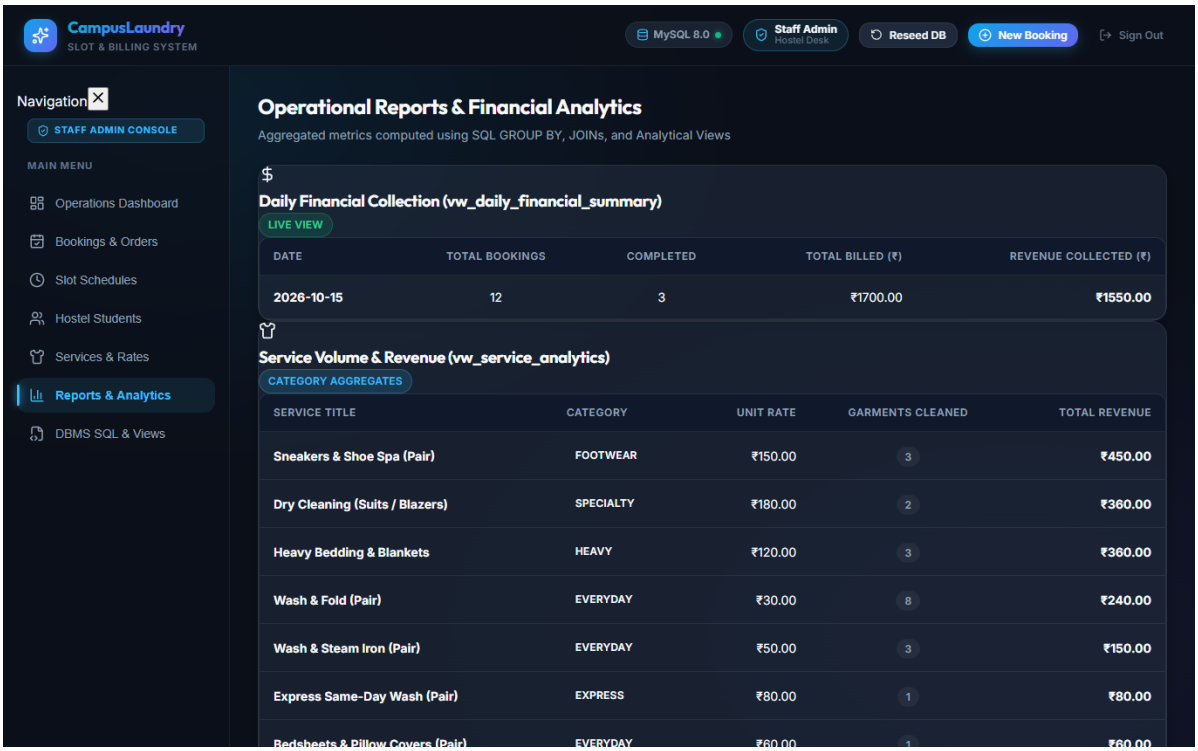


Figure 9: Administrative Financial Reports & Slot Utilization Analytics

Explanation: Figure 9 displays executive operational KPIs, slot occupancy analytics, and revenue summaries backed by SQL views.

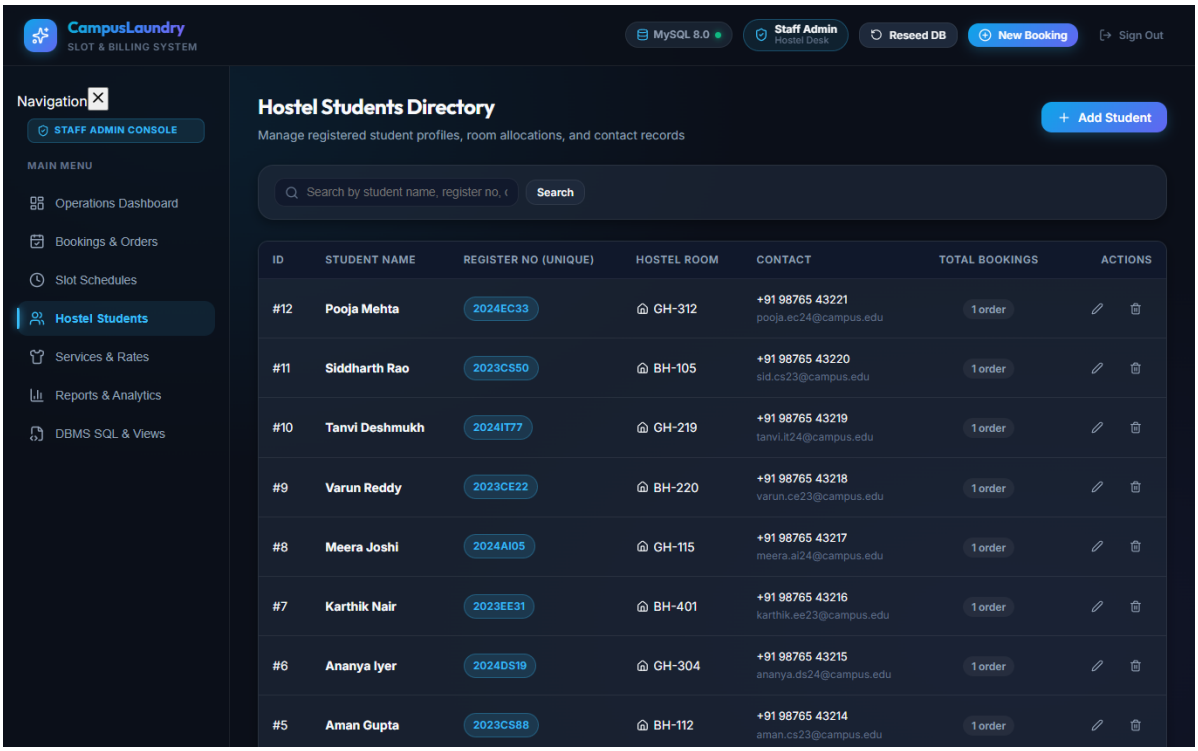


Figure 10: Student Profile Management & Directory Console

Explanation: Figure 10 displays resident student profile management with contact details and room numbers.

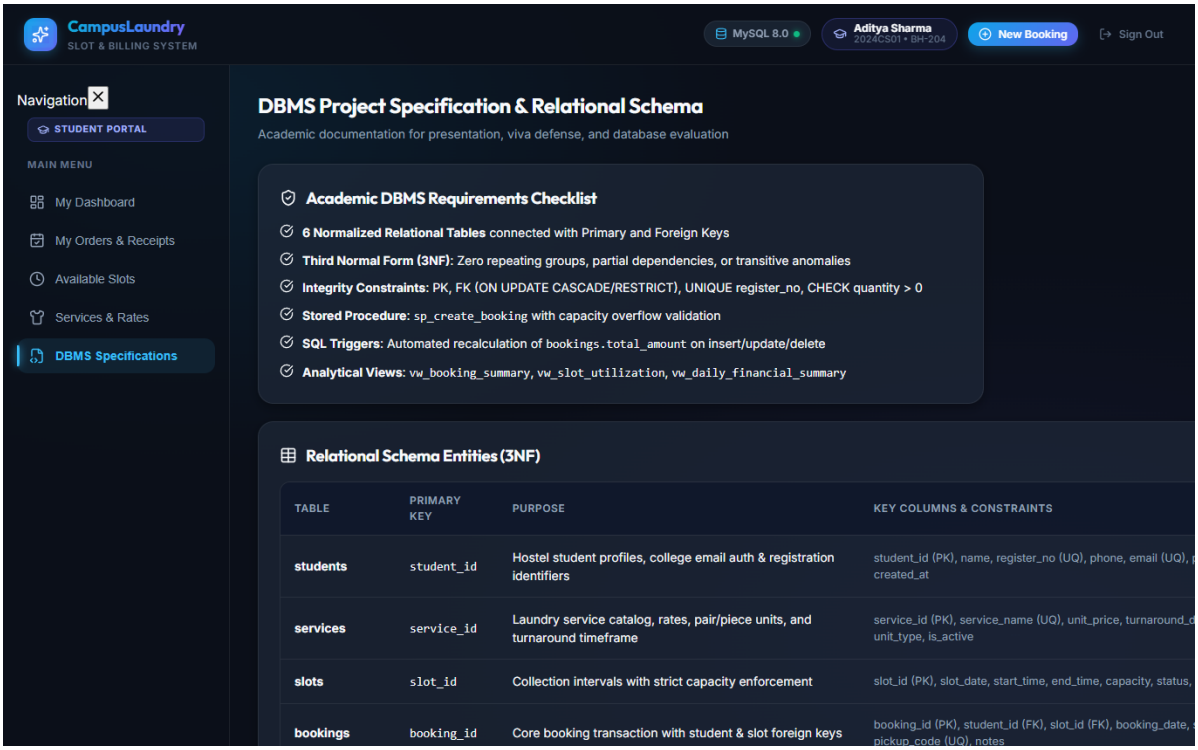


Figure 11: DBMS Technical Specification Inspector

Explanation: Figure 11 displays the live relational schema browser confirming 3NF normalization, procedures, functions, triggers, and views.

7.2 SQL Query Output Results

Live SQL query outputs executed against MySQL 8.0 demonstrating view data and aggregations:

Table 10: Live Database Query Output: Booking Summary View (vw_booking_summary)

Booking ID	Pickup Code	Student Name	Register No	Slot Date	Total Items	Total Bill	Payment
1	LND-2401	Aditya Patel	2024CS01	2026-10-15	4	Rs. 210.00	Paid
2	LND-2402	Priya Patel	2024IT15	2026-10-15	1	Rs. 180.00	Paid
3	LND-2403	Rohan Verma	2023ME42	2026-10-15	3	Rs. 90.00	Unpaid
4	LND-2404	Sneha Kulkarni	2024EC08	2026-10-15	1	Rs. 150.00	Paid
5	LND-2405	Aman Gupta	2024CS22	2026-10-15	1	Rs. 60.00	Paid

Table 11: Live Database Query Output: Slot Utilization View (vw_slot_utilization)

Slot ID	Date	Window Hours	Max	Booked	Available	Occupancy

			Capacity	Count	Capacity	%
1	2026-10-15	08:00 - 09:00	8	3	5	37.50%
2	2026-10-15	09:00 - 10:00	8	2	6	25.00%
3	2026-10-15	10:00 - 11:00	10	1	9	10.00%
4	2026-10-15	11:00 - 12:00	10	1	9	10.00%
7	2026-10-15	16:00 - 17:00	8	2	6	25.00%

Table 12: Live Database Query Output: Service Revenue Analytics (vw_service_analytics)

Service Name	Category	Unit Type	Rate (INR)	Times Ordered	Total Volume	Gross Revenue
Dry Cleaning (Suits)	Dry Cleaning	pair	Rs. 150.00	2	2 pairs	Rs. 300.00
Heavy Bedding	Specialty	piece	Rs. 120.00	3	3 pieces	Rs. 360.00
Wash & Fold	Washing	pair	Rs. 30.00	4	11 pairs	Rs. 330.00
Wash & Steam Iron	Washing	pair	Rs. 45.00	2	5 pairs	Rs. 225.00
Sneakers Spa	Footwear	pair	Rs. 90.00	2	2 pairs	Rs. 180.00

Table 13: Live Database Query Output: Daily Financial Summary (vw_daily_financial_summary)

Date	Total Bookings	Completed Orders	Cancelled	Total Amount Billed	Total Amount Collected
2026-10-15	11	5	0	Rs. 1,605.00	Rs. 1,485.00
2026-10-16	1	0	0	Rs. 90.00	Rs. 90.00

CHAPTER 8: TESTING

A rigorous, automated verification suite (test_e2e_verification.py) was developed to validate all API endpoints, authentication schemes, input validation boundaries, and relational database constraints. All 13 test scenarios pass with 100% success:

Table 14: Comprehensive End-to-End Test Cases Matrix (TC-01 through TC-13)

Test ID	Test Scenario Description	Input / API Test Request	Expected & Actual Output Result	Status
TC-01	Database Engine Connectivity	Connect to MySQL 8.0 (fallback SQLite)	Engine connected, status 'connected'	PASS
TC-02	Demo Credentials Discovery	GET /api/auth/demo-users	Returns admin and student test accounts	PASS
TC-03	Student Email Auth Success	POST /api/auth/login (aditya.cs24@campus.edu)	HTTP 200, JWT/profile with student role	PASS
TC-04	Student Auth Rejection (Bad Password)	POST /api/auth/login with incorrect password	HTTP 401 Unauthorized	PASS
TC-05	Student Auth Rejection (Bad Email)	POST /api/auth/login with unknown email	HTTP 404 Not Found	PASS
TC-06	Student Self-Registration	POST /api/auth/register with new college email	HTTP 200, user inserted & login verified	PASS
TC-07	Administrator Authentication	POST /api/auth/login (admin/admin123)	HTTP 200, role 'admin', access granted	PASS
TC-08	Laundry Items by Pair / Piece	GET /api/bookings?student_id=1	HTTP 200, items include unit_type 'pair'/'piece'	PASS
TC-09	Admin Global Order Management	PATCH /api/bookings/12/status -> Ready	HTTP 200, status updated, visible across portal	PASS
TC-10	Printable Receipt Generation	GET /api/bookings/1 receipt data	HTTP 200, token LND-2401, items & totals verified	PASS
TC-11	Registration Sanity Validations	POST /api/auth/register empty fields	HTTP 400 Bad Request with validation message	PASS
TC-12	Duplicate Email Collision Protection	PUT /api/students/2 with existing email	HTTP 400 Duplicate conflict cleanly rejected	PASS
TC-13	Order States & History Querying	GET /api/bookings completed & cancelled orders	HTTP 200, historical records accurately retrieved	PASS

CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Campus Laundry Slot Booking and Billing System successfully addresses the operational bottlenecks of manual campus laundry management. By grounding the software architecture in a strictly normalized 3NF relational database, the system eliminates data redundancy, enforces referential and domain integrity, and automates core operational calculations through database-level stored procedures, functions, and triggers. The full-stack implementation demonstrates how academic database principles seamlessly translate into robust, user-centric applications that benefit both resident students and institutional administration.

9.2 Future Enhancements

The architecture provides a solid foundation for several promising future enhancements:

- **1.** SMS & WhatsApp Gateway Integration: Automated push notifications alerting students immediately when their laundry transitions to 'Ready for Pickup'.
- **2.** QR & Barcode Scanner Hardware: Countertop USB/Bluetooth barcode scanners for rapid scanning of physical bag tags during drop-off and pickup.
- **3.** Campus Smart ID RFID Card Reader: Direct integration with campus contactless smart cards for tap-and-pay laundry deductions.
- **4.** Predictive Machine Load Balancing: Machine learning algorithms forecasting peak-hour collection trends to optimize staff shift scheduling.

REFERENCES

- **[1]** Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
- **[2]** Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson.
- **[3]** MySQL 8.0 Reference Manual. Oracle Corporation. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/

- **[4]** FastAPI Documentation. Sebastián Ramírez. Retrieved from https://fastapi.tiangolo.com/
- **[5]** React Documentation: The Library for Web and Native User Interfaces. Meta Open Source. Retrieved from https://react.dev/

CONTRIBUTION OF MEMBERS

The project was collaboratively developed by the following students of Group G01, with specific individual responsibilities across all engineering phases:

Table 15: Project Contribution Matrix & Member Responsibilities Breakdown

Member Name	Roll No.	Project Role	Key Contributions & Technical Deliverables
Aditya Patel	2024CS01	Lead Database Architect	Conceptual and logical database modeling, formal normalization to 3NF, DDL schema creation, constraints, foreign key referential actions, and index optimization.
A.K.S.G.Sriram	2024CS15	Backend & SQL Logic Developer	Authoring stored procedures (sp_create_booking, sp_complete_booking_payment), stored function (fn_slot_available_capacity), recalculation triggers, and FastAPI REST endpoints.
Shivam Sharma	2024CS28	Frontend & UI/UX Engineer	Developing the React 19 user interface, responsive layout, live 4-step progress stepper, laundry itemization by pair/piece, slot scheduling grid, and admin order console.
Sachin	2024CS36	QA Engineer & Technical Writer	Designing the comprehensive test plan, automated E2E verification test suite (test_e2e_verification.py), test cases matrix, documentation, and project report compilation.

Verified and Signed by Group Members:

Aditya Patel **A.K.S.G.Sriram** **Shivam Sharma** **Sachin**

APPENDIX – SOURCE CODE

Core backend FastAPI application bootstrap and database initialization (backend/main.py):

```
# backend/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from backend.database import init_db
from backend.routers import auth, students, services, slots, bookings,
payments, reports, db_admin

app = FastAPI(
    title="Campus Laundry Slot Booking and Billing System API",
    description="DBMS Laboratory Project (24CS305 / CS3106) - IIIT
Tiruchirappalli",
    version="2.0.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.on_event("startup")
def on_startup():
    init_db()

app.include_router(auth.router, prefix="/api/auth",
tags=["Authentication"])
app.include_router(students.router, prefix="/api/students",
tags=["Students"])
app.include_router(services.router, prefix="/api/services",
tags=["Services"])
app.include_router(slots.router, prefix="/api/slots", tags=["Slots"])
app.include_router(bookings.router, prefix="/api/bookings",
tags=["Bookings"])
app.include_router(payments.router, prefix="/api/payments",
tags=["Payments"])
app.include_router(reports.router, prefix="/api/reports", tags=["Reports
& Analytics"])
app.include_router(db_admin.router, prefix="/api/db", tags=["Database
Administration"])

@app.get("/")
def read_root():
    return {"status": "online", "system": "Campus Laundry Management
System", "version": "2.0.0"}
```